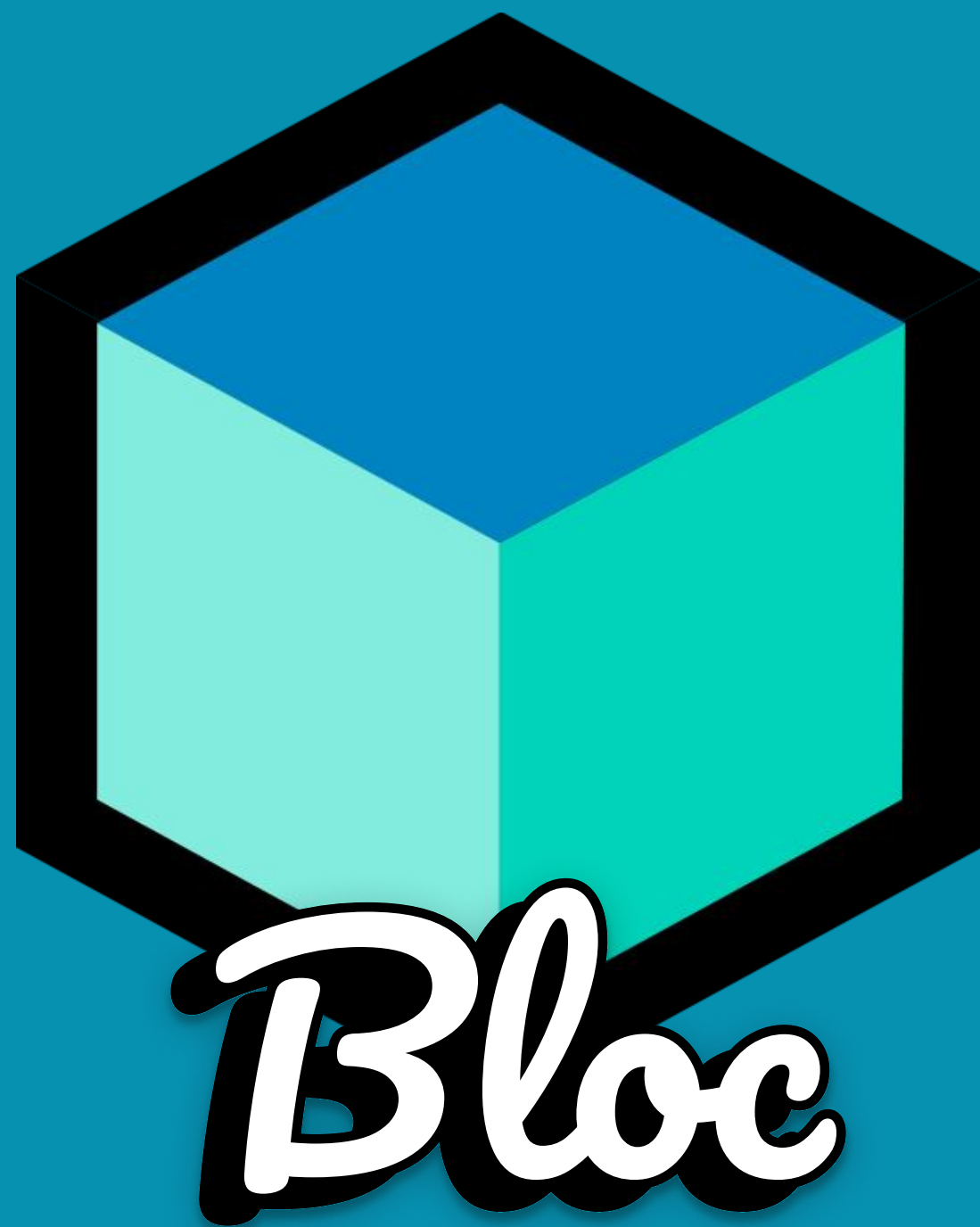
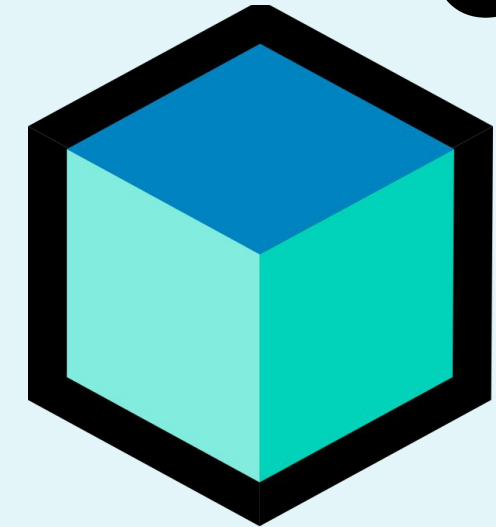




About the Package

What does the acronym BLoC mean?

B: **Business**, Lo: **Logic**, C: **Component**



What is BLoC?

It is a **Pattern** that aims to **separate business logic from the user interface**.

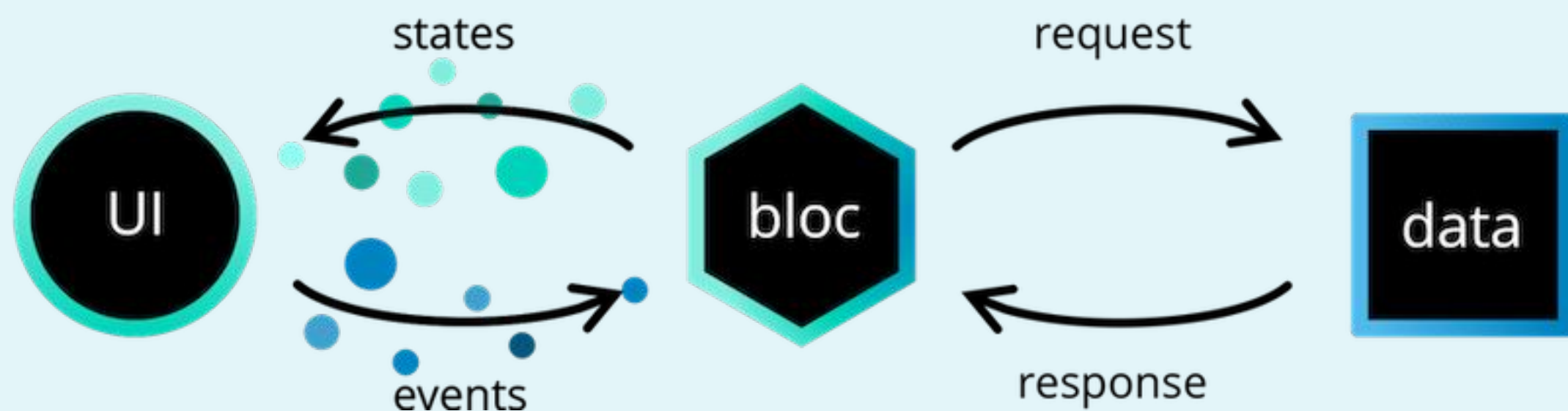
What reactivity does the package use?

Stream, being a single-valued reactivity.

Is BLoC a silver bullet?

No. For simpler functionality cases, ChangeNotifier and ValueNotifier are easier and are native to Flutter.

How does it work?



We will work with 3 classes:

Events: These are **user actions** or other changes that can affect the state of the application.

States: Representation of the **current status of the application**.

Bloc: Where the **reactivity will be executed**. Receiving events, it executes the business logic and changes the state depending on the response and notifies whoever is listening.

Data input and output in streams are done in different places, as if it were a straw.

Polymorphism

One factor that makes beginners find it difficult to understand BLoC is not understanding one of the pillars of Object Orientation, **Polymorphism**.

It occurs when we use a **common base class** (abstract class) for different types of states or events, the BLoC deals with these classes that inherit from this base class **without needing to know specifically what type of event or state is being processed**.

When inheriting from an abstract class, **the subclass is treated as being of the type of that base class**. The pattern we will use explained in the paragraph above is known as the **State Pattern**.

Contextualizing

I will show in practice the use of bloc by creating a small zip code search application using the famous API: ViaCep.

If the zip code exists, it will return: the street, state and region, and if it does not exist, it will return a message saying that it does not exist.

The image displays three sequential screens of a mobile application titled "VIA CEP". Each screen features a light pink background and a white input field with the placeholder text "Enter the CEP". A blue "Search" button is positioned below the input field. The first screen shows the input field with "0/8" characters. The second screen shows the input field with "01001000" and "8/8" characters, with a light green box below the search button displaying the results: "CEP: 01001-000", "ROAD: Praça da Sé", "STATE: São Paulo", and "REGION: Sudeste". The third screen shows the input field with "00000000" and "8/8" characters, with a bold black message "CEP Not Found" below the search button.

Events

```
sealed class CepEvents extends Equatable {  
    @override  
    List<Object?> get props => [];  
}
```

```
class FetchCepEvent extends CepEvents {  
    final CepModel cepModel;  
  
    FetchCepEvent(this.cepModel);  
  
    @override  
    List<Object?> get props => [cepModel];  
}
```

Equatable: Package that compares the values of the class attributes, because if they are equal it will point to the already instantiated reference.

States

```
sealed class CepStates extends Equatable {  
    @override  
    List<Object?> get props => [];  
}
```

```
class InitialCepState extends CepStates {}
```

```
class SuccessCepState extends CepStates {  
    final CepModel model;  
  
    SuccessCepState(this.model);  
  
    @override  
    List<Object?> get props => [model];  
}
```

```
class FailureCepState extends CepStates {  
    final String message;  
  
    FailureCepState(this.message);  
  
    @override  
    List<Object?> get props => [message];  
}
```

Bloc

In the Block class we define the input and output StreamControllers, which are essential for the application's **reactivity** and **allow events to be sent asynchronously to multiple listeners**.

```
final _inputStream = StreamController<CepEvents>();  
final _outputStream = StreamController<CepStates>();  
  
Sink<CepEvents> get input => _inputStream.sink;  
Stream<CepStates> get stream => _outputStream.stream;
```

Every time the page triggers an event, this method I created: **_blocEventController** will be executed.

```
CepBloc() {  
  _inputStream.stream.listen(_blocEventController);  
}
```

Bloc

This method will handle the events triggered by the page, process the business logic and trigger the events.

```
void _blocEventController(CepEvents event) async {  
  
  if (event is FetchCepEvent) {  
    try {  
      final endpoint = generateEndpointUtils(event.cepModel.cep);  
      final json = await dio.get(endpoint);  
      final result = CepModel.fromMap(json.data);  
  
      _emit(SuccessCepState(result));  
    } catch (e) {  
      _emit(FailureCepState('CEP Not Found'));  
    }  
  }  
}
```

Method to improve visibility.

```
void _emit(CepStates state) => _outputStream.add(state);
```

View

For my page **to listen** to this Bloc class we need to use the **StreamBuilder**.

```
-StreamBuilder<CepStates>(  
  stream: controller.stream,  
  builder: (context, snapshot) {  
    CepStates? state = snapshot.data;  
  
    return switch (state) {  
      SuccessCepState(:final model) =>  
        CardResultCustom(cepModel: model),  
      FailureCepState(:final message) => Text(  
        message,  
        style: TextStyle.h1,  
      ), // Text  
      InitialCepState() => const SizedBox.shrink(),  
      _ => const SizedBox.shrink()  
    };  
  },  
) // StreamBuilder
```

View

And finally, triggering events for the Bloc.

```
-ButtonCustom(  
  onTap: () => fetchCep(cepModel),  
) // ButtonCustom
```

```
void fetchCep(CepModel model) {  
  controller.input.add(FetchCepEvent(model));  
}
```

Refactoring StreamController to Flutter_Bloc (package)

Before

```
class CepBloc {
  CepBloc() {
    _inputStream.stream.listen(_blocEventController);
  }

  final dio = Dio(BaseOptions(baseUrl: apiUrl));

  final _inputStream = StreamController<CepEvents>();
  final _outputStream = StreamController<CepStates>();

  Sink<CepEvents> get input => _inputStream.sink;
  Stream<CepStates> get stream => _outputStream.stream;

  void _blocEventController(CepEvents event) async {
    if (event is FetchCepEvent) {
      try {
        final endpoint = generateEndpointUtils(event.cepModel.cep);
        final json = await dio.get(endpoint);
        final result = CepModel.fromMap(json.data);

        _emit(SuccessCepState(result));
      } catch (e) {
        _emit(FailureCepState('CEP Not Found'));
      }
    }
  }

  void _emit(CepStates state) => _outputStream.add(state);
}
```

After

Much simpler

```
class CepFlutterBloc extends Bloc<CepEvents, CepStates> {  
  CepFlutterBloc() : super(InitialCepState()) {  
    //  
    final dio = Dio(BaseOptions(baseUrl: apiUrl));  
  
    on<FetchCepEvent>((event, emit) async {  
      try {  
        final endpoint = generateEndpointUtils(event.cepModel.cep);  
        final json = await dio.get(endpoint);  
        final result = CepModel.fromMap(json.data);  
  
        emit(SuccessCepState(result));  
      } catch (e) {  
        emit(FailureCepState('CEP Not Found'));  
      }  
    });  
  }  
}
```

Before

```
-StreamBuilder<CepStates>(
|   stream: controller.stream,
|   builder: (context, snapshot) {
|       CepStates? state = snapshot.data;
|
|       return switch (state) {
|           SuccessCepState(:final model) =>
|               CardResultCustom(cepModel: model),
|           FailureCepState(:final message) => Text(
|               message,
|               style: TextStyle.h1,
|           ), // Text
|           InitialCepState() => const SizedBox.shrink(),
|           _ => const SizedBox.shrink()
|       };
|   },
| ) // StreamBuilder
```

After

Almost nothing has changed

```
-BlocBuilder<CepFlutterBloc, CepStates>(
  bloc: controller,
  builder: (context, state) {
    return switch (state) {
      SuccessCepState(:final model) =>
        CardResultCustom(cepModel: model),
      FailureCepState(:final message) => Text(
        message,
        style: TextStyle.h1,
      ), // Text
      InitialCepState() => const SizedBox.shrink()
    };
  },
) // BlocBuilder
```

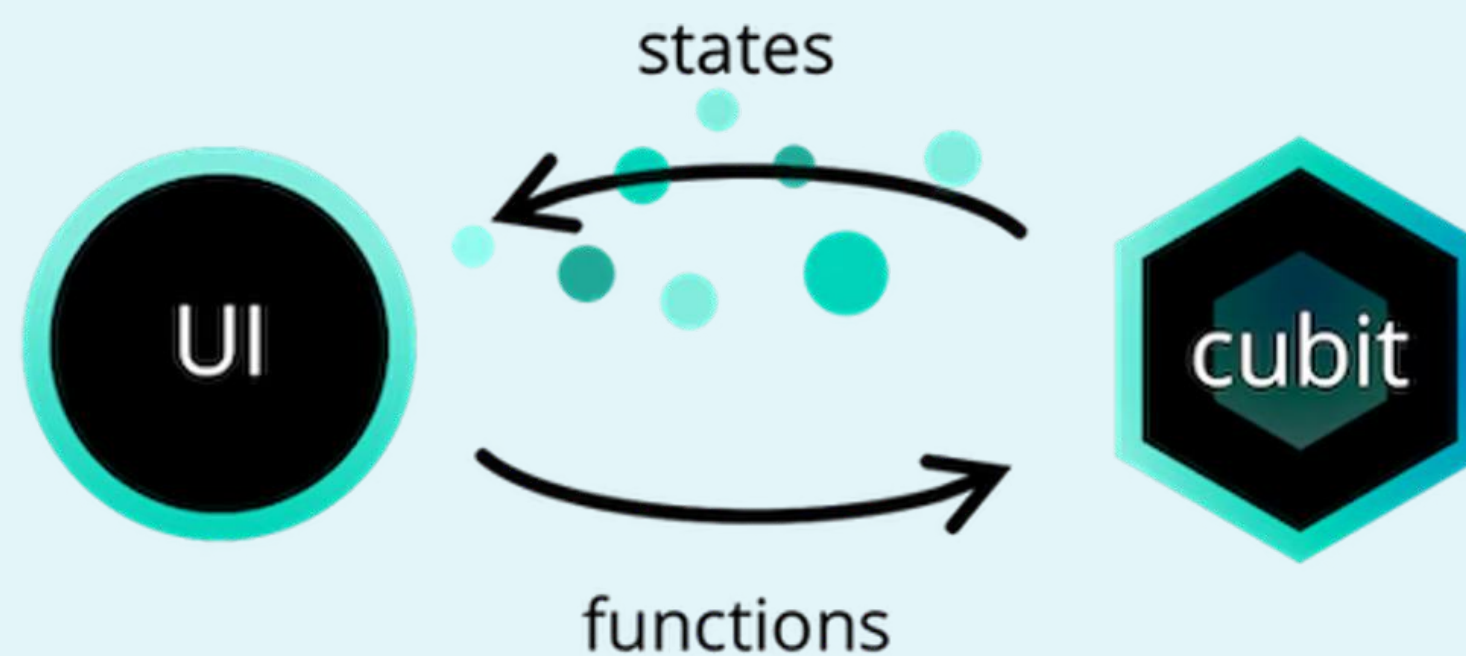


What about Cubit?

Cubit

It's almost the same thing as ValueNotifier, except the difference is that **one** is a **Listenable** and the other is a **Stream**.

What is the difference between Cubit and Bloc?



- BLoC: Receives events and emits states.
- Cubit: Emits states.